BOB-AGS: Business Operations Butler — Call Center AI Dashboard

> **Live Monitoring Edition** — Real-time call analysis and SME guidance for substance abuse helpline agents

---

Table of Contents

1. Project Overview
2. System Architecture
3. Tech Stack
4. Directory Structure
5. Authentication
6. CTM Integration
7. Live Monitoring System (SME Mode)
8. AI Scoring System
9. RAG Knowledge Base
10. Key API Routes
11. Database Schema
12. Environment Variables

---

1. Project Overview

BOB (Business Operations Butler) is a Next.js call tracking and AI-powered quality assurance dashboard for substance abuse helpline calls. It integrates with **CallTrackingMetrics (CTM)** for call data and **AssemblyAI** for real-time transcription, with AI scoring via **OpenRouter**.

The system operates in two primary modes:

Mode	Purpose	Key Technology
Post-Call Analysis	Score and review recorded calls	AssemblyAI REST API → OpenRouter AI

Mode	Purpose	Key Technology
Live Monitoring (SME Mode)	Real-time guidance during active calls	AssemblyAI WebSocket streaming → dual AI analyzer

Live Monitoring as an SME

When agents are on live calls, BOB acts as a **Subject Matter Expert (SME)** — providing real-time prompts, warnings, and disposition suggestions based on a 25-criterion QA rubric. It detects:

- Crisis language (suicide, overdose, relapse)
- Compliance violations (HIPAA, unqualified transfers, medical advice)
- Missing qualification steps (insurance verification, state, sobriety)
- Suggested next actions based on detected context

---

2. System Architecture

High-Level Data Flow

```
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                                LIVE CALL MONITORING                                  │
│                                                                                      │
├──────────────────────────────────────────────────────────────────────────────────────┤
│  Agent Phone → Microphone → AudioWorklet/ScriptProcessor → AssemblyAI RT WS          │
│                                                                                      │
│                                                ↓                                     │
│                                                                                      │
│                                AssemblyAI Realtime Transcriber                       │
│                                                                                      │
│                                                ↓                                     │
│                                                                                      │
│   ┌──────────────────────┐     ┌──────────────────────────────────────────────┐     │
│                                                                                      │
│   │  Keyword Analyzer    │     │        OpenRouter (Claude 3 Haiku)           │     │
│                                                                                      │
│   │ (analyzer.ts)        │     │  useLiveAIInsights (15s throttle)            │     │
│                                                                                      │
│   │                      │     │                                              │     │
│                                                                                      │
```

```
│  │ • Pass/fail phrases │    │ • Crisis detection                          │
│
│  │ • Score update      │    │ • Disposition suggestion                    │
│
│  │ • ZTP auto-fail     │    │ • Agent guidance                            │
│
│  └─────────────────────┘    └─────────────────────────────────────────────┘
│
│           ↓                                  ↓
│
│  ┌─────────────────────┐    ┌─────────────────────────────────────────────┐
│
│  │  LiveInsightsPanel  │    │     LiveAIInsightsPanel /                   │
│
│  │  (realtime rubric)  │    │     KnowledgeSuggestions (RAG)              │
│
│  └─────────────────────┘    └─────────────────────────────────────────────┘
│
└──────────────────────────────────────────────────────────────────────────────
┘

┌──────────────────────────────────────────────────────────────────────────────
┐
│                              POST-CALL ANALYSIS
│
├──────────────────────────────────────────────────────────────────────────────
┤
│  CTM API → /api/ctm/calls/[id]/audio → AssemblyAI REST (upload + poll)
│
│                                          ↓
│
│  AssemblyAI Transcript → /api/openrouter → OpenRouter GPT-4o-mini
│
│                                          ↓
│
│  25-Criterion Rubric Evaluation → Score → Tags → Disposition
│
└──────────────────────────────────────────────────────────────────────────────
┘
```

Auth Flow

```
Login → /api/auth/login → Supabase Auth → Set session cookie
  ↓
Middleware (proxy.ts, Edge Runtime) → Refresh session on every request
```

```
  ↓
Protected routes → getSession() validates cookie → Return data or 401
```

---

3. Tech Stack

Layer	Technology	Version
Framework	Next.js	16.1.6 (App Router, Turbopack)
Language	TypeScript	Strict mode
UI	React	19.2.4
Styling	Tailwind CSS 4	+ shadcn/ui (Radix primitives)
Auth	Supabase SSR	`@supabase/ssr`
Database	Supabase	PostgreSQL
Call Data	CallTrackingMetrics	REST API (Basic Auth)
Transcription	AssemblyAI	REST API + WebSocket streaming
AI Scoring	OpenRouter	Claude 3 Haiku / GPT-4o-mini
Testing	Playwright	`@playwright/test`
Package Manager	pnpm	Node 22.x
Deployment	Vercel	Fluid Compute

---

4. Directory Structure

```
bob-ags-nextjs/
├── app/                          # Next.js App Router
│   ├── api/                      # API routes (server-side)
│   │   ├── admin/                # Fix user role, import agents
│   │   ├── agents/profiles/      # Agent profile lookups
│   │   ├── assemblyai/           # Token + transcription proxy
│   │   ├── auth/                 # Login, logout, session, forgot-password
│   │   ├── calls/                # Notes, QA overrides
│   │   ├── ctm/                  # CTM proxy routes
│   │   │   ├── active-calls/     # Live call polling
│   │   │   ├── calls/            # List, search, bulk-sync, analyze
│   │   │   ├── agents/           # Agent list + groups
│   │   │   ├── numbers/          # Number search + purchase
```

```
│   │   │   ├── dashboard/stats/  # Dashboard KPIs
│   │   │   └── live-calls/       # CTM live call data
│   │   ├── knowledge-base/       # RAG knowledge base endpoint
│   │   ├── live-analysis-logs/   # Supabase log storage
│   │   ├── openrouter/          # AI analysis proxy
│   │   └── users/               # Permissions, settings, CTM assignments
│   ├── auth/                     # Auth pages (login, signup, reset)
│   ├── dashboard/               # Protected dashboard pages
│   │   ├── agents/             # Agent management
│   │   ├── calls/              # Call listing + detail
│   │   ├── history/            # Search + CSV export
│   │   ├── monitor/            # Live call monitoring
│   │   ├── qa-logs/           # QA analysis history
│   │   └── settings/           # User credentials + preferences
│   ├── page.tsx                # Login page (root)
│   ├── layout.tsx              # Root layout
│   └── globals.css             # CSS variables + Tailwind
│
├── components/                   # React components
│   ├── ui/                      # shadcn/ui base (Button, Card, Dialog, etc.)
│   ├── dashboard/               # Dashboard shared components
│   ├── call-detail/             # Call detail page components
│   ├── monitor/                 # Live monitoring components
│   ├── settings/                # Settings page components
│   ├── agents/                  # Agent management components
│   └── history/                # History page components
│
├── hooks/                        # Custom React hooks
│   ├── dashboard/               # useCalls, useDashboard, useAgents, etc.
│   ├── monitor/                # useLiveAnalysis, useLiveAIInsights,
useMonitorPage
│   ├── calls/                  # useCallDetail
│   ├── rag/                    # useKnowledgeBase
│   └── settings/               # useSettings
│
├── lib/                          # Core business logic
│   ├── ai/                      # AI scoring (api-client, rubric, scoring,
etc.)
│   ├── ctm/                    # CTM API client + services factory
│   ├── realtime/               # AssemblyAI WebSocket streaming
│   │   ├── assemblyai-realtime.ts  # Streaming transcriber class
│   │   ├── analyzer.ts           # Keyword-pattern rubric analyzer
│   │   ├── audio-processor.ts     # AudioWorklet + downsampling
│   │   ├── constants.ts           # RUBRIC_KEYWORDS, patterns
│   │   └── types.ts              # RealtimeTranscript, RealtimeInsight
│   ├── rag/                    # RAG knowledge base + suggestions
```

```
│   ├── supabase/               # Supabase client helpers
│   ├── calls/                  # Call data fetching + transformation
│   ├── types/                  # Shared TypeScript types
│   └── utils/                  # Formatters, helpers
│
├── supabase/                     # Supabase configuration
│   ├── migrations/              # Database schema migrations
│   └── functions/              # Edge Functions (on-auth-signup)
│
├── docs/                         # Project documentation
│   ├── AI_SCORING_SYSTEM.md    # 25-criterion rubric, ZTP, dispositions
│   ├── CTM_API_DOCUMENTATION.md
│   ├── 02_PRE_RECORDED_AUDIO.md
│   └── 03_STREAMING_AUDIO.md
│
├── public/
│   └── audio-worklet-processor.js  # Browser AudioWorklet processor
│
├── middleware.ts                  # Edge Runtime auth middleware
├── next.config.mjs              # Next.js config
├── tailwind.config.ts           # Tailwind CSS 4 config (navy scale)
└── package.json
```

---

5. Authentication

Dual Auth System

Two systems coexist:

System	Purpose	Cookie
Supabase SSR	Primary auth for all users	`sb-session` (HTTP-only)
Legacy HMAC	Developer bypass for local dev	`sb-dev-session`

Supabase Server Client Pattern

```
// In API routes — use the proxy pattern:
const { supabase, response } = await createServerSupabase(request)
return response
```

Critical Session Rule

Always use `getSession()` (not `getUser()`) to refresh cookies on API routes. Using `getUser()` alone causes `getSession()` to return null.

Dev Bypass

The `sb-dev-session` cookie allows local development without real Supabase auth. When present, it's converted to a fake `sb-session` placeholder so downstream Supabase client code works normally.

Dev credentials: `agsdev@allianceglobalsolutions.com` / `ags2026@@`

Middleware

Located at `proxy.ts` in the project root (Edge Runtime middleware):

- Runs at the Edge on every request
- Creates a Supabase server client and refreshes sessions
- Matcher excludes `_next/static`, `_next/image`, favicon, and static file extensions

---

6. CTM Integration

Base Client

`lib/ctm/client.ts` — `CTMClient` base class with:

- Basic Auth: `Authorization: Basic base64(ACCESS_KEY:SECRET_KEY)`
- API base: `https://api.calltrackingmetrics.com/api/v1`
- Rate limit: 1000 req/min

Service Factory Pattern

```
createCTMClient(config)        // Creates configured CTMClient
createCallsService(client)     // Calls: list/detail/transcript/bulk-sync
createAgentsService(client)    // Agents: profiles/groups
createNumbersService(client)   // Numbers: search/purchase
createReceivingNumbersService(client)
createSchedulesService(client)
createSourcesService(client)
createVoiceMenusService(client)
createAccountsService(client)
```

CTM API Proxy Routes (`app/api/ctm/`)

All CTM routes proxy requests to hide credentials:

Route	Purpose
`GET /api/ctm/active-calls`	List active calls (polled every 2s during monitoring)
`GET /api/ctm/calls`	List calls with filters
`GET /api/ctm/calls/[id]`	Call detail
`GET /api/ctm/calls/[id]/transcript`	Fetch transcript from CTM
`GET /api/ctm/calls/[id]/audio`	Fetch call recording audio
`POST /api/ctm/calls/analyze`	Bulk analyze (AssemblyAI → OpenRouter)
`GET /api/ctm/agents`	List agents
`GET /api/ctm/agents/groups`	Agent groups
`GET /api/ctm/numbers/search`	Search available numbers
`POST /api/ctm/numbers/purchase`	Purchase a number
`GET /api/ctm/dashboard/stats`	Dashboard KPIs

---

7. Live Monitoring System (SME Mode)

This is the core SME (Subject Matter Expert) functionality — real-time call analysis and guidance during live calls.

Architecture Overview

```
Microphone → AudioWorklet → 48kHz → downsampler → 16kHz Int16 → AssemblyAI
WebSocket
                                                                    ↓
                                                  ┌──────────────────────────────┘
                                                  ↓
                                     AssemblyAI StreamingTranscriber
                                                  ↓ `turn` events

┌────────────────────────┴────────────────────────┐
                                ↓
↓
                    analyzeTextForInsights()
useLiveAIInsights hook
                    (keyword matching, instant)
```

```
(OpenRouter, 15s throttle)
                              ↓
↓
                    LiveInsightsPanel
LiveAIInsightsPanel
                    (rubric pass/fail/warning)                    (crisis,
disposition, guidance)
                              ↓
                    ScoreProgress + CriteriaStatus
                              ↓
                    KnowledgeSuggestions (RAG)
                    (criterion-matched suggestions)
```

Components on Monitor Page

The monitor page (`app/dashboard/monitor/page.tsx`) has two columns:

Left column (7/12 width):

- `MonitorHeader` — Start/Stop controls, call info, duration timer
- `TranscriptPanel` — Dual-column Agent / Caller transcript, auto-scrolling
- `LiveInsightsPanel` — Real-time rubric pass/fail/warning insights by category
- `LiveAIInsightsPanel` — AI suggestions, crisis alerts, disposition

Right column (5/12 width):

- `NotesDispositionPanel` — Notes + disposition
- `CallDetailsCard` — Phone, state, insurance, caller info
- `ScoreProgress` — Progress bar showing criteria completion (X/25)
- `KnowledgeSuggestions` — RAG-powered contextual suggestions

Floating panel:

- `LiveAnalysisLogHistory` — Collapsible, bottom-right

AssemblyAI WebSocket Streaming (`lib/realtime/assemblyai-realtime.ts`)

`AssemblyAIRealtime` **class:**

```
class AssemblyAIRealtime {
  connect(): Promise<void>
  disconnect(): void
  // Events: 'turn', 'open', 'close', 'error'
```

```
  // State: isConnected, isRecording, duration, transcript[],
  //        insights[], sentiment, criteriaStatus, score
}
```

Connection flow:

1. Request microphone via `navigator.mediaDevices.getUserMedia`
2. Fetch token from `/api/assemblyai/token`
3. Create `AudioContext` at 48kHz, resume it
4. Instantiate `StreamingTranscriber`:
   - `sampleRate: 16000`
   - `speechModel: 'u3-rt-pro'`
   - `speakerLabels: true`
   - `maxSpeakers: 2`
   - `prompt: 'This is a phone call. There are two speakers...'`

Audio pipeline:

- `AudioWorkletNode` (preferred) or `ScriptProcessorNode` (fallback)
- 48kHz microphone input → downsampled to 16kHz via `downsample()`
- Converted to `Int16Array` via `toInt16Array()`
- Buffered in 20ms intervals via `flushAudioBuffer()` → `transcriber.sendAudio(buffer)`

Browser requirement: AudioWorklet requires HTTPS or localhost

Real-Time Analyzer (`lib/realtime/analyzer.ts`)

`analyzeTextForInsights()` **function** — keyword-pattern rubric analyzer:

```
// RUBRIC_KEYWORDS in constants.ts — pass/fail phrases per criterion
const RUBRIC_KEYWORDS = {
  '1.1': { pass: ['hello flyland', 'flyland this is'], fail: ['hi there'] },
  '1.2': { pass: ["what's your name", 'can i get your name'], fail: [] },
  '3.4': { pass: [], fail: ['transfers state insurance', 'transfers self-
pay'], ztp: true },
  '5.1': { pass: [], fail: ['shares info unauthorized', 'hipaa breach'], ztp:
true },
  '5.2': { pass: [], fail: ['detox advice', 'withdrawal advice', 'dosage'],
ztp: true },
  // ... 25 criteria total
}
```

Pattern detection:

- `INSURANCE_PATTERNS` — Medicaid, Medicare, Tricare, Kaiser, Private, Self-pay
- `STATE_PATTERNS` — All 50 US states via regex
- `NAME_PATTERNS` — Caller name extraction
- `POSITIVE_WORDS` / `NEGATIVE_WORDS` — Sentiment

Insights generated:

```
interface RealtimeInsight {
  type: 'pass' | 'fail' | 'warning' | 'info'
  criterion: string        // e.g. '1.1', '3.4'
  criterionId: string
  category: string         // Opening, Probing, Qualification, etc.
  message: string
  timestamp: number
  autoFail?: boolean
  ztp?: boolean
}
```

ZTP Auto-fail: Criteria 3.4, 5.1, 5.2 immediately set score to 0 when fail phrase is detected.

AI Live Insights (`hooks/monitor/useLiveAIInsights.ts`)

OpenRouter (Claude 3 Haiku) for higher-level analysis:

Triggers:

- Transcript has 50+ characters
- Throttled to once per 15 seconds (`ANALYSIS_INTERVAL`)

Output format:

```
{
  "insights": [{
    "type": "suggestion|warning|disposition",
    "title": "...",
    "message": "...",
    "priority": "high|medium|low"
  }],
  "suggestedDisposition":
"qualified|not_qualified|follow_up|transfer_988|unclassified"
}
```

Crisis detection (`checkCrisis()`):
| Detected phrase | Action |

|----------|--------|
`suicide` / `suicidal`	Crisis Detected → Transfer to 988
`kill myself` / `going to die`	Suicidal intent escalation
`overdose` / `od'd` / `took too many`	Medical emergency → 911
`relapse` / `started using again`	Relapse support

System prompt role: "AI assistant for a substance abuse helpline agent" providing real-time guidance, NOT QA evaluation.

Monitor Page Hook (`hooks/monitor/useMonitorPage.ts`)

Polling:

- Polls `/api/ctm/active-calls` every 2 seconds
- Stops polling while actively monitoring (to save resources)

Auto-start for viewers:

- When a new call arrives for a viewer with an agent assignment, plays a beep and auto-starts monitoring
- `hasAutoStartedRef` flag prevents multiple auto-starts

Grace period:

- When monitored call ends, 30-second countdown (`GRACE_PERIOD_SECONDS`)
- After countdown, monitoring stops automatically

Audio Processor (`lib/realtime/audio-processor.ts`)

Handles browser audio capture:

```
// Primary: AudioWorkletNode (low latency, modern browsers)
// Fallback: ScriptProcessorNode (compatibility)
class AudioProcessor {
  start(stream: MediaStream): void
  stop(): void
  // Emits buffer events at ~20ms intervals
}
```

Key Files

File	Purpose
`app/dashboard/monitor/page.tsx`	Main monitor UI

File	Purpose
`lib/realtime/assemblyai-realtime.ts`	AssemblyAI WebSocket class
`lib/realtime/analyzer.ts`	Keyword-pattern rubric analyzer
`lib/realtime/audio-processor.ts`	AudioWorklet + ScriptProcessorNode fallback
`lib/realtime/constants.ts`	RUBRIC_KEYWORDS, INSURANCE_PATTERNS, STATE_PATTERNS
`hooks/monitor/useLiveAnalysis.ts`	React hook wrapping AssemblyAIRealtime
`hooks/monitor/useLiveAIInsights.ts`	OpenRouter AI analysis hook
`hooks/monitor/useMonitorPage.ts`	Call polling, auto-start, grace period
`public/audio-worklet-processor.js`	Browser AudioWorklet processor

---

8. AI Scoring System

The 25-Criterion QA Rubric

Defined in `lib/ai/rubric.ts` — **107 points maximum** across 5 categories:

Category	Max Points	Criteria
Opening	14 pts	1.1 Greeting (2), 1.2 Name (2), 1.3 Reason (5), 1.4 State (5)
Probing	19 pts	2.1 Sobriety (5), 2.2 Substance (5), 2.3 Insurance (5), 2.4 Additional info (2), 2.5 Phone (2)
Qualification	37 pts	3.1 Eligibility (5), 3.2 Needs (5), 3.3 Rebuttals (5), 3.4 Unqualified transfer** (5), 3.5 Escalation (5), 3.6 Referrals (5), 3.7 Empathy (2)
Closing	14 pts (3 N/A)	4.1 Professional (2), 4.2 Salesforce (N/A), 4.3 Star rating (N/A), 4.4 Follow-up (N/A)
Compliance	26 pts	5.1 HIPAA **(5), 5.2 Medical advice** (5), 5.3 Response time (2), 5.4 Soft skills (2), 5.5 SOP (5)

** = ZTP (Zero Tolerance Policy) — auto-fail if violated

Zero Tolerance Policy (ZTP)

Three criteria trigger **immediate auto-fail (score = 0)** regardless of other criteria:

ID	Criterion	Fail Phrases
3.4	Avoided unqualified transfers	"transfers state insurance", "transfers self-pay", "unqualified transfer"
5.1	Upheld HIPAA	"shares info unauthorized", "hipaa breach", "unauthorized disclosure"
5.2	Avoided medical advice	"detox advice", "withdrawal advice", "dosage", "treatment recommendation"

Scoring Formula

```
if (autoFailed || ztpFailures >= 2) return 0
const totalMax = 107  // evaluable points only
const totalScore = sum of all category scores
return Math.round((totalScore / totalMax) * 100)
```

Dispositions

Score	Disposition	CTM Star Rating
80–100	**Qualified Lead**	4 stars
60–79	**Warm Lead**	3 stars
40–59	**Refer** (SAMHSA/988)	2 stars
0–39	**Do Not Refer**	1 star
Auto-fail	**Critical Violation** (supervisor review)	0 stars

Tags Generated

Tag	Condition
`excellent`	score >= 85
`good`	score >= 70
`needs-improvement`	score >= 50
`poor`	score < 50
`{category}-gap`	Any failed criterion in that category
`unqualified-transfer`	Criterion 3.4 failed
`hipaa-risk`	Criterion 5.1 failed

Tag	Condition
`medical-advice-risk`	Criterion 5.2 failed
`ztp-violation`	Any ZTP criterion failed
`insurance:{type}`	Detected insurance type
`state:{state}`	Detected US state

Analysis Flow

1. **Transcript** submitted to `lib/ai/api-client.ts` (`analyzeTranscript()`)
2. **OpenAI GPT-4o-mini** via `OPENAI_API_KEY` evaluates against rubric
3. **Response** parsed in `CRITERION_ID|PASS/FAIL/N/A|reason` format
4. **Fallback** to keyword matching if AI returns nothing for a criterion
5. **Score** calculated from rubric breakdown
6. **Tags + disposition** generated from `generation.ts`

Note: The codebase references `OPENROUTER_API_KEY` in documentation, but the actual implementation uses `OPENAI_API_KEY` with GPT-4o-mini. The OpenRouter route (`app/api/openrouter/route.ts`) proxies to OpenAI.

Key Files

File	Purpose
`lib/ai/rubric.ts`	RUBRIC_CRITERIA, ZTP_CRITERIA, ALWAYS_NA_CRITERIA, CATEGORY_WEIGHTS
`lib/ai/api-client.ts`	`analyzeTranscript()` — OpenAI API call
`lib/ai/prompt-builder.ts`	Build GPT prompt for rubric evaluation
`lib/ai/result-parser.ts`	Parse AI responses (pipe-delimited format)
`lib/ai/scoring.ts`	`evaluateRubric()` , `calculateBreakdown()` , `calculateScore()`
`lib/ai/keyword-matching.ts`	`keywordMatch()` fallback — pass phrases vs fail phrases
`lib/ai/generation.ts`	`generateTags()` , `generateDisposition()` , `generateSummary()`
`lib/ai/extraction.ts`	`extractNames()` , `detectInsurance()` , `detectCallType()`
`app/api/openrouter/route.ts`	POST endpoint — calls `analyzeTranscript()`

9. RAG Knowledge Base

The system includes a **Retrieval-Augmented Generation (RAG)** knowledge base that provides contextual suggestions to agents during live calls, mapped to the 25-criterion QA rubric.

Knowledge Entries (`lib/rag/knowledge-base.ts`)

Category	Count	Content
`transfer`	9	Medicaid (SAMHSA), Medicare, Private insurance, Self-pay, 988 Lifeline, Family/Al-Anon, state-specific resources
`script`	8	Approved greeting, Insurance verification, Sobriety question, State verification, Reason for call, Substance inquiry, Transfer script, Non-qualifying callers
`sop`	5	HIPAA compliance, No medical advice, Unqualified transfer prevention, Documentation requirements, Empathy standards
`disposition`	4	Qualified (85-100), Warm lead (50-84), Informational (0-49), ZTP supervisor review

Suggestions Mapped to Criteria (`lib/rag/suggestions.ts`)

Each criterion ID maps to a suggestion with title and message:

```
const SUGGESTIONS: Record<string, Suggestion> = {
  '1.1': {
    title: 'Use Approved Greeting',
    message: 'Open with: "Hello Flyland, this is [name]. How can I help you
today?"'
  },
  '2.3': {
    title: 'Verify Insurance',
    message: 'Ask: "Do you have insurance, and if so, what type?"'
  },
  '3.4': {
    title: 'CRITICAL: No Unqualified Transfers',
    message: 'Do NOT transfer Medicaid/Medicare callers to treatment
centers...'
  },
  // ... all 25 criteria
}
```

Relevance Scoring (`getRelevantKnowledge()`)

Match Type	Score Boost
Insurance match	+10
State match	+5
Crisis match	+10
Caller type match	+10
Missing criteria match	+15

Hooks

```
useKnowledgeBase(category)   // Fetch entries by category
useSearchKnowledge(query)    // Search all entries
```

UI Component (`components/monitor/KnowledgeSuggestions.tsx`)

- Searchable knowledge base with collapsible category sections
- Color-coded badges: transfer (cyan), script (navy), sop (amber), disposition (emerald)
- Displays criterion-linked suggestions alongside live insights

API Endpoint

`GET /api/knowledge-base?q={query}&category={category}` — Returns relevant entries with relevance scores.

---

10. Key API Routes

Authentication

Route	Method	Purpose
`/api/auth/login`	POST	Supabase login
`/api/auth/logout`	POST	Clear session
`/api/auth/session`	GET	Get current session
`/api/auth/agent-lookup`	GET	Look up agent by email

CTM Proxy

Route	Method	Purpose
`/api/ctm/active-calls`	GET	Polled active calls (2s interval)
`/api/ctm/calls`	GET	List calls
`/api/ctm/calls/[id]`	GET	Call detail
`/api/ctm/calls/history`	GET	Call history
`/api/ctm/calls/search`	GET	Search calls
`/api/ctm/calls/bulk-sync`	POST	Bulk sync calls
`/api/ctm/calls/analyze`	POST	Analyze call (AssemblyAI → OpenRouter)
`/api/ctm/agents`	GET	List agents
`/api/ctm/agents/groups`	GET	Agent groups
`/api/ctm/numbers/search`	GET	Search numbers
`/api/ctm/numbers/purchase`	POST	Purchase number
`/api/ctm/dashboard/stats`	GET	Dashboard KPIs

AI & Transcription

Route	Method	Purpose
`/api/assemblyai/token`	GET	Get AssemblyAI token
`/api/assemblyai/transcribe`	POST	Transcribe pre-recorded audio
`/api/openrouter`	POST	AI analysis of transcript
`/api/live-analysis-logs`	GET/POST	Live analysis log history

Call Notes & QA

Route	Method	Purpose
`/api/calls/[id]/notes`	GET/POST	Call notes
`/api/qa-overrides`	GET/POST	QA score overrides

---

11. Database Schema

Key Supabase tables:

Table	Purpose
`users`	Authentication
`user_roles`	Role definitions (admin, qa, agent, viewer)
`agent_profiles`	CTM agent ID → Supabase user mapping
`user_settings`	Per-user CTM/OpenRouter API credentials
`live_analysis_logs`	Live monitoring session history
`qa_overrides`	Manual QA score overrides

Key Migrations

```sql
-- Agent profile mapping
CREATE TABLE agent_profiles (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES auth.users,
  ctm_agent_id TEXT,
  ctm_agent_name TEXT
);

-- Live analysis log
CREATE TABLE live_analysis_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users,
  call_id TEXT,
  transcript TEXT,
  score INTEGER,
  disposition TEXT,
  insights JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- QA overrides
CREATE TABLE qa_overrides (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  call_id TEXT UNIQUE,
  original_score INTEGER,
  override_score INTEGER,
  reason TEXT,
  override_by UUID REFERENCES auth.users,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

12. Environment Variables

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=

# Session
NEXTAUTH_SECRET=
NEXTAUTH_URL=

# CTM (CallTrackingMetrics)
CTM_ACCESS_KEY=
CTM_SECRET_KEY=
CTM_ACCOUNT_ID=

# AI
OPENAI_API_KEY=           # Primary — used in lib/ai/api-client.ts
OPENROUTER_API_KEY=        # Referenced in settings (not used in
analyzeTranscript)

# AssemblyAI
ASSEMBLYAI_API_KEY=
```

Key Pages

Path	Purpose
`/`	Login page (demo + dev credentials)
`/dashboard`	Stats overview + recent calls
`/dashboard/calls/[id]`	Call detail: score, transcript, AI insights
`/dashboard/monitor`	Live call monitoring with SME guidance
`/dashboard/history`	Search + CSV export
`/dashboard/settings`	Store API credentials

Demo Credentials

Type	Email	Password
Demo	`demo@example.com`	`demo`
Dev	`agsdev@allianceglobalsolutions.com`	`ags2026@@`

Generated: 2026-04-10